

HW9

2025-04-28

Problem 1 #part a

```
simulated_a <- function(n = 500, nsims = 1000, seed = 567) {  
  set.seed(seed)  
  out <- matrix(NA, nrow = nsims, ncol = 6,  
               dimnames = list(NULL,  
                                c("tau_true", "tau_dm", "tau_reg", "tau_int", "tau_proj", "tau_aipw_dd")))  
  
  for(i in seq_len(nsims)) {  
    x      <- rnorm(n)  
    eps1   <- rnorm(n)  
    eps2   <- rnorm(n)  
    Y1     <- 1 + x + x^3 + eps1  
    Y0     <- 1 - x + eps2  
    tau_t  <- mean(Y1 - Y0)  
  
    W      <- sample(rep(c(1,0), each = n/2))  
    Y      <- ifelse(W==1, Y1, Y0)  
  
    tau_dm <- mean(Y[W==1]) - mean(Y[W==0])  
  
    X_sc   <- as.numeric(scale(x))  
    X3_sc  <- as.numeric(scale(x^3))  
    df     <- data.frame(Y=Y, W=W, X_sc=X_sc, X3_sc=X3_sc)  
  
    mod_reg <- lm(Y ~ W + X_sc + X3_sc, data = df)  
    tau_reg <- coef(mod_reg)["W"]  
  
    mod_int <- lm(Y ~ W*X_sc + W*X3_sc, data = df)  
    tau_int <- coef(mod_int)["W"]  
  
    mu1_hat <- predict(mod_int,  
                      newdata = transform(df, W=1))  
    mu0_hat <- predict(mod_int,  
                      newdata = transform(df, W=0))  
    tau_proj <- mean(mu1_hat - mu0_hat)  
  
    p_hat   <- mean(W)  
    dr_term <- W*(Y - mu1_hat)/p_hat - (1-W)*(Y - mu0_hat)/(1-p_hat)  
    tau_aipw <- mean(mu1_hat - mu0_hat) + mean(dr_term)  
  
    out[i,] <- c(tau_t, tau_dm, tau_reg, tau_int, tau_proj, tau_aipw)  
  }  
}
```

```

out_df <- as.data.frame(out)
summary <- data.frame(
  estimator = names(out_df),
  mean      = sapply(out_df, mean),
  variance  = sapply(out_df, var)
)
list(raw = out_df, summary = summary)
}

res_a <- simulated_a()
print(res_a$summary[-1, ])

```

```

##              estimator      mean  variance
## tau_dm        tau_dm -0.009303516 0.09828552
## tau_reg        tau_reg  0.001352274 0.06152278
## tau_int        tau_int -0.000645141 0.06614836
## tau_proj       tau_proj -0.000645141 0.06614836
## tau_aipw_dd    tau_aipw_dd -0.000645141 0.06614836

```

#part b

```

simulated_b <- function(n = 20, nsims = 1000, seed = 567) {
  set.seed(seed)
  out <- matrix(NA, nrow = nsims, ncol = 6,
    dimnames = list(NULL,
      c("tau_true", "tau_dm", "tau_reg", "tau_int", "tau_proj", "tau_aipw_dd")))

  for(i in seq_len(nsims)) {
    x <- rnorm(n)
    e <- rexp(n)
    Y1 <- 1 + x + x^2 + 4*x^3 + x^4 + e
    Y0 <- -Y1
    tau_t <- mean(Y1 - Y0)

    W <- sample(rep(c(1,0), each = n/2))
    Y <- ifelse(W==1, Y1, Y0)
    tau_dm <- mean(Y[W==1]) - mean(Y[W==0])

    f1 <- as.numeric(scale(x))
    f2 <- as.numeric(scale(x^2))
    f3 <- as.numeric(scale(x^3))
    f4 <- as.numeric(scale(x^4))
    df <- data.frame(Y=Y, W=W, f1=f1, f2=f2, f3=f3, f4=f4)

    mod_reg <- lm(Y ~ W + f1 + f2 + f3 + f4, data = df)
    tau_reg <- coef(mod_reg)["W"]

    mod_int <- lm(Y ~ W*(f1+f2+f3+f4), data = df)
    tau_int <- coef(mod_int)["W"]

    mu1_hat <- predict(mod_int, newdata = transform(df, W=1))
    mu0_hat <- predict(mod_int, newdata = transform(df, W=0))
    tau_proj <- mean(mu1_hat - mu0_hat)
  }
}

```

```

p_hat    <- mean(W)
dr_term  <- W*(Y - mu1_hat)/p_hat - (1-W)*(Y - mu0_hat)/(1-p_hat)
tau_aipw <- mean(mu1_hat - mu0_hat) + mean(dr_term)

out[i,] <- c(tau_t, tau_dm, tau_reg, tau_int, tau_proj, tau_aipw)
}

out_df <- as.data.frame(out)
summary <- data.frame(
  estimator = names(out_df),
  mean      = sapply(out_df, mean),
  variance  = sapply(out_df, var)
)
list(raw = out_df, summary = summary)
}

res_b <- simulated_b()
print(res_a$summary[-1, ])

```

```

##              estimator      mean  variance
## tau_dm        tau_dm -0.009303516 0.09828552
## tau_reg        tau_reg  0.001352274 0.06152278
## tau_int        tau_int -0.000645141 0.06614836
## tau_proj       tau_proj -0.000645141 0.06614836
## tau_aipw_dd    tau_aipw_dd -0.000645141 0.06614836

```

#part c I used the following DGP : $X_i \stackrel{iid}{\sim} U(0,1)$, $Y_i(1) = 2 + 3X_i + \epsilon_{1i}$, and $Y_i(0) = 1 + \epsilon_{2i}$, where $\epsilon_{1i} \stackrel{iid}{\sim} N(0,1)$ and $\epsilon_{2i} \stackrel{iid}{\sim} N(0,1)$, sample size is $n = 200$.

```

simulated_c <- function(n = 200, nsims = 1000, seed = 567) {
  set.seed(seed)
  out <- matrix(NA, nrow = nsims, ncol = 6,
    dimnames = list(NULL,
      c("tau_true", "tau_dm", "tau_reg", "tau_int", "tau_proj", "tau_aipw_dd")))

  for(i in seq_len(nsims)) {
    x    <- runif(n)
    eps1 <- rnorm(n)
    eps2 <- rnorm(n)
    Y1   <- 2 + 3*x + eps1
    Y0   <- 1 + eps2
    tau_t <- mean(Y1 - Y0)

    W    <- sample(rep(c(1,0), each = n/2))
    Y    <- ifelse(W==1, Y1, Y0)
    tau_dm <- mean(Y[W==1]) - mean(Y[W==0])

    X_sc <- as.numeric(scale(x))
    df   <- data.frame(Y=Y, W=W, X_sc=X_sc)

    mod_reg <- lm(Y ~ W + X_sc, data = df)
    tau_reg <- coef(mod_reg)["W"]
  }
}

```

```

mod_int <- lm(Y ~ W*X_sc, data = df)
tau_int <- coef(mod_int)["W"]

mu1_hat <- predict(mod_int, newdata = transform(df, W=1))
mu0_hat <- predict(mod_int, newdata = transform(df, W=0))
tau_proj <- mean(mu1_hat - mu0_hat)

p_hat <- mean(W)
dr_term <- W*(Y - mu1_hat)/p_hat - (1-W)*(Y - mu0_hat)/(1-p_hat)
tau_aipw <- mean(mu1_hat - mu0_hat) + mean(dr_term)

out[i,] <- c(tau_t, tau_dm, tau_reg, tau_int, tau_proj, tau_aipw)
}

out_df <- as.data.frame(out)
summary <- data.frame(
  estimator = names(out_df),
  mean      = sapply(out_df, mean),
  variance  = sapply(out_df, var)
)
list(raw = out_df, summary = summary)
}

res_c <- simulated_c()
print(res_a$summary[-1, ])

```

```

##           estimator      mean  variance
## tau_dm      tau_dm -0.009303516 0.09828552
## tau_reg      tau_reg  0.001352274 0.06152278
## tau_int      tau_int -0.000645141 0.06614836
## tau_proj      tau_proj -0.000645141 0.06614836
## tau_aipw_dd tau_aipw_dd -0.000645141 0.06614836

```

Problem 2 #part a In an experiment, you design the study. you decide who gets the treatment and who doesn't, randomization is present. So, all covariates are clean and can be associated with treatment or control. In an observational study, you just get the data where someone else performed the experiment, so you can't be sure of randomization and can't ignore confounding. So, it get difficult as we use different statistical methods to gain covariate balance, making the causal inference harder compared to an experiment.

Problem 3

```
library(designmatch)
```

```
## Loading required package: lattice
```

```
## Loading required package: MASS
```

```
## Loading required package: slam
```

```
## Loading required package: highs
```

```
##
## Thank you for using our package! To acknowledge our work, please cite the package as:

## Zubizarreta, Jose R., Kilcioglu, Cinar, Vielma, Juan P., and Cohn, Eric R. (2023). designmatch: Mat

## R package version 0.5.2. https://cran.r-project.org/web/packages/designmatch/
```

```
library(cobalt)
```

```
## cobalt (Version 4.6.0, Build Date: 2025-04-15)
```

```
library(boot)
```

```
##
## Attaching package: 'boot'

## The following object is masked from 'package:lattice':
##
##      melanoma
```

```
library(ggplot2)
```

```
data(lalonde, package = "designmatch")
d <- lalonde

Y <- d$re78
W <- d$treatment
X <- subset(d, select = -c(re78, treatment))
head(d)
```

```
##      treatment age education black hispanic married nodegree re74 re75      re78
## 1           1  37         11     1         0         1         1  0  0 9930.0460
## 2           1  22          9     0         1         0         1  0  0 3595.8940
## 3           1  30         12     1         0         0         0  0  0 24909.4500
## 4           1  27         11     1         0         0         1  0  0  7506.1460
## 5           1  33          8     1         0         0         1  0  0   289.7899
## 6           1  22          9     1         0         0         1  0  0 4056.4940
```

#part a We need to make same assumptions as we did in problem 2. Assumptions : SUTVA, strong ignorability $Y(W) \perp W \mid X$ for $W = 0,1$, and overlap $0 < P(W = 1 \mid X) < 1$.

#part b

```
ps_mod <- glm(treatment ~ age + education + black + hispanic +
              married + nodegree + re74 + re75,
              data = d, family = binomial)

new_person <- data.frame(
  age      = 22,
  education = 12,
  black    = 0,
```

```

  hispanic = 0,
  married  = 0,
  nodegree = 0,
  re74     = 1000,
  re75     = 0
)
ps_hat_person <- predict(ps_mod, newdata = new_person, type = "response")
cat("Estimated Propensity score :", round(ps_hat_person,4), "\n\n")

```

```
## Estimated Propensity score : 0.0746
```

```

ps_hat <- predict(ps_mod, type = "response")
ipw_wt <- ifelse(W == 1, 1/ps_hat, 1/(1 - ps_hat))
haj_wt <- ipw_wt

```

Estimated propensity score of 0.0746 means that, someone with same covaraites (an individual who is 22, has 12 years of education and a high school diploma, is not black nor hispanic, is not married, and earned \$1000 in 1974 and \$0 in 1975) has about 7.46% chance of being given the treatment.

#part c

```

ipw_est <- mean(W * Y / ps_hat) - mean((1-W) * Y / (1-ps_hat))
cat("Estimate =", round(ipw_est,2), "\n")

```

```
## Estimate = -449.79
```

```

boot_ipw <- function(data, indices) {
  d0 <- data[indices, ]
  ps <- predict(ps_mod, newdata = d0, type = "response")
  with(d0,
    mean(treatment * re78 / ps) -
    mean((1-treatment) * re78 / (1-ps)))
}
set.seed(123)
B <- 1000
boot_ipw_out <- boot(data = d, statistic = boot_ipw, R = B)
ipw_var <- var(boot_ipw_out$t)
ipw_ci <- ipw_est + c(-1,1) * 1.96 * sqrt(ipw_var)

```

```

## Warning in c(-1, 1) * 1.96 * sqrt(ipw_var): Recycling array of length 1 in vector-array arithmetic i
## Use c() or as.vector() instead.

```

```
cat("\n", "Estimated Variance =", round(ipw_var,2), "\n")
```

```

##
## Estimated Variance = 1167886

```

```
cat("95% CI= [", round(ipw_ci[1],2), ", ", round(ipw_ci[2],2), "]\n\n")
```

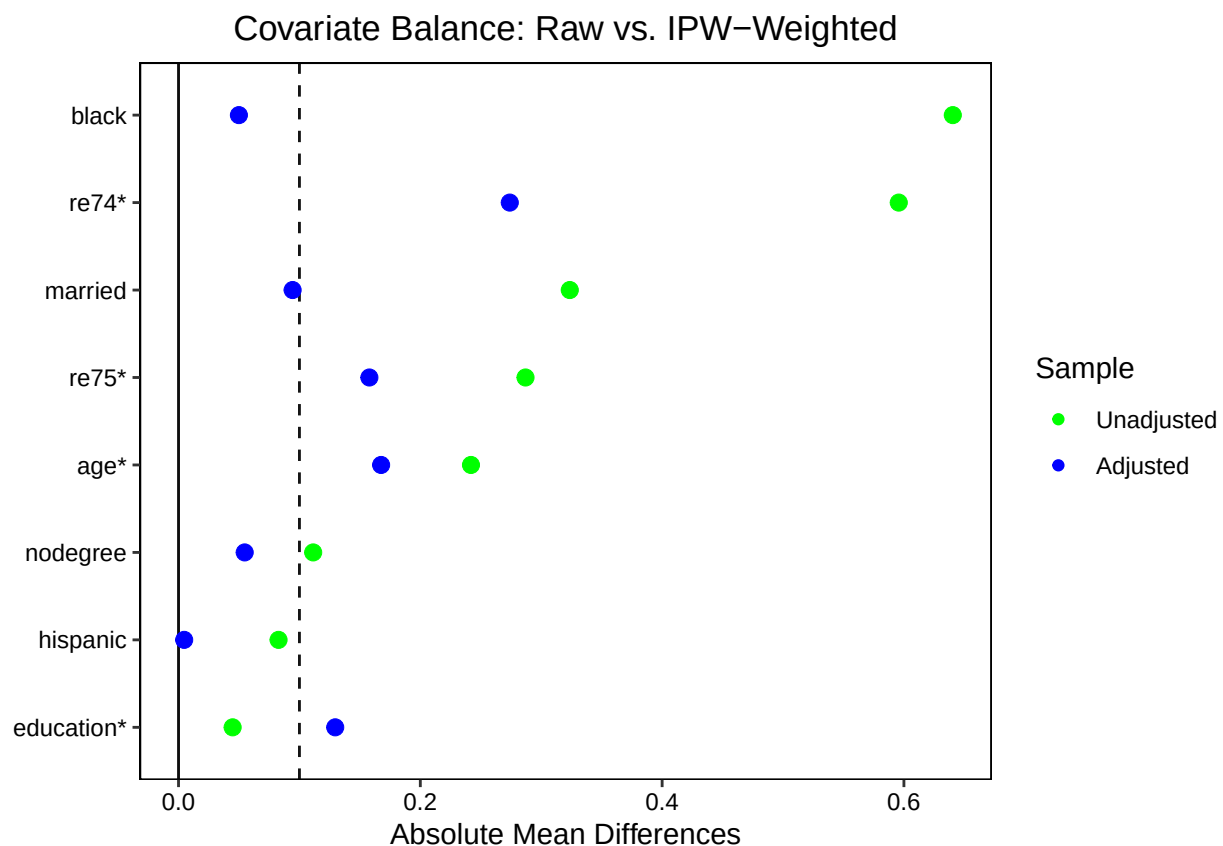
```
## 95% CI= [ -2567.94 , 1668.36 ]
```

#part d

```
bal_ipw <- bal.tab(  
  treatment ~ age + education + black + hispanic +  
    married + nodegree + re74 + re75,  
  data      = d,  
  weights   = ipw_wt,  
  method    = "weighting",  
  un        = TRUE,  
  quick     = FALSE  
)
```

Note: 's.d.denom' not specified; assuming "pooled".

```
lp <- love.plot(  
  bal_ipw,  
  var.order = "unadjusted",  
  stats     = "mean.diffs",  
  abs       = TRUE,  
  thresholds = c(m = .1),  
  colors    = c("green", "blue"),  
  stars     = "std",  
  title     = "Covariate Balance: Raw vs. IPW-Weighted"  
)  
print(lp)
```



After adjusting, the mean differences (blue) are closer to the zero line compared to unadjusted (green) indicating reduced covariant imbalance. But there still seems to be imbalance, which needs to be addressed.

#part e

```
tau_t_num <- sum(W * Y / ps_hat)
tau_t_den <- sum(W / ps_hat)
tau_c_num <- sum((1-W) * Y / (1 - ps_hat))
tau_c_den <- sum((1-W) / (1 - ps_hat))
haj_est <- tau_t_num/tau_t_den - tau_c_num/tau_c_den
cat("Estimate=", round(haj_est,2), "\n")
```

Estimate= 224.68

```
boot_haj <- function(data, indices) {
  d0 <- data[indices, ]
  ps <- predict(ps_mod, newdata = d0, type = "response")
  num_t <- sum(d0$treatment * d0$re78 / ps)
  den_t <- sum(d0$treatment / ps)
  num_c <- sum((1-d0$treatment) * d0$re78 / (1-ps))
  den_c <- sum((1-d0$treatment) / (1-ps))
  num_t/den_t - num_c/den_c
}
set.seed(123)
boot_haj_out <- boot(data = d, statistic = boot_haj, R = B)
haj_var <- var(boot_haj_out$t)
haj_ci <- haj_est + c(-1,1) * 1.96 * sqrt(haj_var)
```

Warning in c(-1, 1) * 1.96 * sqrt(haj_var): Recycling array of length 1 in vector-array arithmetic is deprecated. Use c() or as.vector() instead.

```
cat("\n", "Estimated Variance=", round(haj_var,2), "\n")
```

##

Estimated Variance= 810853.8

```
cat("95% CI= [", round(haj_ci[1],2), ", ", round(haj_ci[2],2), "]\n\n")
```

95% CI= [-1540.25 , 1989.61]

#part f

```
ps_trunc <- pmax(pmin(ps_hat, .95), .05)

ipw_est_tr <- mean(W * Y / ps_trunc) - mean((1-W) * Y / (1-ps_trunc))
boot_ipw_tr <- function(data, indices) {
  d0 <- data[indices, ]
  ps <- predict(ps_mod, newdata = d0, type = "response")
  ps <- pmax(pmin(ps, .95), .05)
  with(d0,
    mean(treatment * re78 / ps) - mean((1-treatment) * re78 / (1-ps)))
}
set.seed(123)
```



```

boot_ipw_tr_out <- boot(data = d, statistic = boot_ipw_tr, R = B)
ipw_var_tr <- var(boot_ipw_tr_out$t)

haj_est_tr <- {
  num_t <- sum(W * Y / ps_trunc); den_t <- sum(W / ps_trunc)
  num_c <- sum((1-W) * Y / (1-ps_trunc)); den_c <- sum((1-W) / (1-ps_trunc))
  num_t/den_t - num_c/den_c
}

boot_haj_tr <- function(data, indices) {
  d0 <- data[indices, ]
  ps <- predict(ps_mod, newdata = d0, type = "response")
  ps <- pmax(pmin(ps, .95), .05)
  num_t <- sum(d0$treatment * d0$re78 / ps); den_t <- sum(d0$treatment / ps)
  num_c <- sum((1-d0$treatment) * d0$re78 / (1-ps)); den_c <- sum((1-d0$treatment) / (1-ps))
  num_t/den_t - num_c/den_c
}

set.seed(123)
boot_haj_tr_out <- boot(data = d, statistic = boot_haj_tr, R = B)
haj_var_tr <- var(boot_haj_tr_out$t)

cat("After truncation\n",
    "IPW Estimate =", round(ipw_est_tr,2), ", estimated variance =", round(ipw_var_tr,2), "\n",
    "Hajek Estimate=", round(haj_est_tr,2), ", estimated variance =", round(haj_var_tr,2), "\n")

## After truncation
## IPW Estimate = -600.04 , estimated variance = 1114773
## Hajek Estimate= 432.17 , estimated variance = 675962.2

```

HW10

Problem 2

b) We need to prove, $E[Y(0)] = E\left[\frac{(1-w)Y}{1-e(x)}\right]$ under strong ignorability, overlap and SUTVA conditions.

WKT, with SUTVA, ignorability & overlap for IPW.

$$z = E[Y(1) - Y(0)] = E\left[\frac{wY}{e(x)} - \frac{(1-w)Y}{1-e(x)}\right]$$

From slide, W.K.T, $E[Y(1)] = E\left[\frac{wY}{e(x)}\right]$

$$\text{So, } E[Y(0)] = E[Y(1)] - z$$

$$\begin{aligned} &= E\left[\frac{wY}{e(x)}\right] - E\left[\frac{wY}{e(x)} - \frac{(1-w)Y}{1-e(x)}\right] \\ &= E\left[\cancel{\frac{wY}{e(x)}} - \cancel{\frac{wY}{e(x)}} + \frac{(1-w)Y}{1-e(x)}\right] \\ &= E\left[\frac{(1-w)Y}{1-e(x)}\right] \end{aligned}$$

c) IPW estimator is,

$$\hat{\tau}_{ipw} = \frac{1}{n} \sum_{i=1}^n \frac{1}{\hat{e}(x_i)} w_i y_i - \frac{1}{n} \sum_{i=1}^n \frac{1}{1-\hat{e}(x_i)} (1-w_i) y_i$$

$$\text{where, } \hat{\tau}_T = \frac{1}{n} \sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)}, \quad \hat{\tau}_C = \frac{1}{n} \sum_{i=1}^n \frac{1-w_i}{1-\hat{e}(x_i)}$$

Say, $y_i^* = y_i + c$

$$\begin{aligned} & \frac{1}{n} \sum_{i=1}^n \frac{1}{\hat{e}(x_i)} w_i y_i^* - \frac{1}{n} \sum_{i=1}^n \frac{1}{1-\hat{e}(x_i)} (1-w_i) y_i^* \\ &= \frac{1}{n} \sum_{i=1}^n \frac{1}{\hat{e}(x_i)} w_i y_i + \left(\frac{1}{n} \sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)} \right) \hat{\tau}_c - \\ & \quad \frac{1}{n} \sum_{i=1}^n \frac{1}{1-\hat{e}(x_i)} (1-w_i) y_i - \\ & \quad \frac{1}{n} \left(\sum_{i=1}^n \frac{1}{1-\hat{e}(x_i)} (1-w_i) \right) c \\ & \quad \hat{\tau}_c \end{aligned}$$

$$= \hat{\tau}_{ipw} + c (\hat{\tau}_T - \hat{\tau}_c)$$

d) Hajek estimator,

$$\hat{\tau}_{haj} = \frac{\sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)} y_i}{\sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)}} - \frac{\sum_{i=1}^n \frac{1-w_i}{1-\hat{e}(x_i)} y_i}{\sum_{i=1}^n \frac{1-w_i}{1-\hat{e}(x_i)}}$$

Say, $y_i^* = y_i + c$

$$\frac{\sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)} (y_i^*)}{\sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)}} - \frac{\sum_{i=1}^n \frac{1-w_i}{1-\hat{e}(x_i)} y_i^*}{\sum_{i=1}^n \frac{1-w_i}{1-\hat{e}(x_i)}}$$

$$= \frac{\sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)} y_i + \sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)} c}{\sum_{i=1}^n \frac{w_i}{\hat{e}(x_i)}}$$

$$+ \frac{\sum_{i=1}^n \frac{(1-w_i)}{1-\hat{e}(x_i)} y_i + \sum_{i=1}^n \frac{(1-w_i)}{1-\hat{e}(x_i)} c}{\sum_{i=1}^n \frac{(1-w_i)}{1-\hat{e}(x_i)}}$$

$$= \hat{\tau}_{haj} + \cancel{c} - \cancel{c}$$

$$= \hat{\tau}_{haj} \quad (\text{remains unchanged when } y_i \rightarrow y_i + c)$$